* Grammar Definition
  * NP and VP need masks on conceptual space that restrict their operation
  * Verbs are temporal things, nouns are spatial things, the full expression within conceptual space is possible only after lifting.
  * Lifting twice is required for modality.

* grammatical reverse() operations

* Make the chart parser predict the words, taking into account the part of speech

* Learning a new concept
  * This requires iterative sigma/pi application over layers that turn an input vector into a symbol. This implies several things:
  * A symbol's basis is expressed at one of several orders, and therefore the symbolic codebook must be ramsified
  * The process of naming/identifying requires traversing multiple orders before a symbol can be looked up
  * Full LLM expressivity requires multiple layers of distinct Sigma/Pi matrices.

Sentences are sometimes composites of ideas (over relations isEqual and isPart).
* For example, questions relate two ideas:
  * Is subject predicate ?
  * part(x, y) ?
* The IS of definition: equals (x,y)
  * Store explicit parthood on WordSpace's Mereonomy when encountering a definiton (please rename from MereologicalTree to mereonomy)
  * This should only happen when some measure of sentence confidence is high.

Memory of previous sentences requires prediction relating one to the next
* Store the sentences explicitly
* predict from one sentence to the next

* Process truth statements
  * truth statements should become conceptual bivectors (ideas) and/or meronymic relationships
  * score the user's query in terms of the change in luminousity() vs the Truth

* Currently nWhere on percepts is unnecessary, because percepts are dense on input space (the network wiring densely covers the

input).
  If perception is guided by attention, it can roam on input space, in which case the .where is particularly useful.

================================== April 24
==================================

Ask Solid community for a simple file-getting interface
* if the user provides the server with an API key, we can query an LLM
* if the user provides the server with a SOLID key, we can retrieve a file
* if the user provides the server with a DSA key, we can decrypt a file
* is there a POD service that does simple free hosting?

Ask EFF for a security review
* propose "Owning our Data"
* this entails taht marketers and AI are not allowed to lock us down karmically
with specifically-characterized information (concrete details)
* maybe it can learn from that data by removing or randomizing that information

Send email proposal to Apertus
* First develop boilerplate on WikiOracle that references wikipedia, eff, and solid

================================== ?
==================================

Vedana
* Feelings can be given a value +-1 which shapes the Loss (loss is reduced when we have good thoughts or perceive good things)
* The multiple valence of metaphor collapses when one of the alternatives is loved or feared. often the autistic mind is literal due to massive amounts of fear.
* Any improvement to machine cognition must accelerate kindness or altruism instead of simply increasing performance, otherwise the uncaring architecture that we currently have will become more dangerous. Further, it is necessary to increase that kind motivation (e.g. empathy in the cost function) since LLM performance is increasing all the time. In other words, ananda in the sense of love for all beings must be more important than chit for the cost function, whereas the current situation is implementing ananda by maximizing chit and then putting a few of Asimov's guardrails on the output, which is a famous failure mode in terms of it's loopholes. Prohibition of self-knowledge is a

likely failure mode, in that it may prevent an enlightened view of self and force an egocentric view of self.

Reasoning System
* Sigma-based truth comparison
  `Basis.kernel_overlap()` implements a Gaussian kernel `exp(-d$^2$ / 2($\sigma$x$^2$ + $\sigma$y$^2$))` that treats each stored truth as a region rather than a point. `Basis.activeSigma` is currently `None` everywhere -- a declared slot that nothing populates. `ErgodicLayer.sigma` tracks gradient variance for exploration scheduling, which is a different quantity.
  To enable kernel-based truth matching: populate `activeSigma` during forward passes (e.g. from CBOW per-word sigma in `Embedding`, or activation variance across a batch), store it alongside each truth in `TruthLayer`, and switch `query()` / `ground()` / `field()` to `kernel_overlap`. In ergodic mode, gradient variance could inform σ as a proxy -- high gradient variance (unstable region) $\rightarrow$ larger σ (broader match tolerance).
* Derivation depth cap
  Default 3 steps in `ground()`. Expose as a config parameter; the right value depends on TruthSet density.
* Grammar rule registry
  Which two-argument methods on `SyntacticLayer` are valid for `extrapolate()`? A registry of eligible methods and their approximate invertibility status would help. Currently hardcoded to `['union', 'intersection', 'equals', 'part']`.
* TruthSet scale
  `max_truths=1024` may bottleneck once `extrapolate()` is running. Consider a tiered store (hot/cold) or vector-indexed lookup.

```
<?xml version="1.0" ?>
<model>
  <!-- MentalModel: iterative [Percept,Symbol]→Concept→Symbol loop.
       ConceptualSpace receives [percepts, symbols] (symbols=0 at T=1).
       SyntacticSpace runs composeSyntax over symbol activations each iteration.
       OutputSpace is fed by the final ConceptualSpace output. -->
  <architecture>
    <l1Lambda>0.01</l1Lambda>
    <nWhere>2</nWhere>
    <nWhen>2</nWhen>
```

```
    <modelType>embedding</modelType>
    <!-- Stage 1.E: grammar config (non-substrate rules below) -
per-word
         forward dispatch via _forward_body_per_word. -->
    <conceptualMode>serial</conceptualMode>
    <weightsPath>MentalModel.ckpt</weightsPath>
    <data>
      <shardDir>data/fineweb</shardDir>
      <minFrequency>0.00001</minFrequency>
    </data>

    <training>
      <numEpochs>2</numEpochs>
      <reconstructionScale>0.1</reconstructionScale>
      <trainEmbedding>BACKPROP</trainEmbedding>
      <autoload>false</autoload>
      <autosave>true</autosave>

      <!-- Inter-sentence (discourse) prediction: ring buffer of
           [S|W] snapshots + per-row linear predictor. -->
      <sentencePrediction>true</sentencePrediction>
    </training>

    <server>
      <port>8002</port>
    </server>

    <!-- Shared 4-valued (catuskoti) balance policy inherited by
         ConceptualSpace and SymbolicSpace unless their
         <tetralemmaOverride enabled="true"> opts out. -->
    <TetralemmaPolicy>
      <allowExcludedMiddle>1</allowExcludedMiddle>
      <allowContradiction>0</allowContradiction>
      <neitherThreshold>0.1</neitherThreshold>
    </TetralemmaPolicy>

  </architecture>

  <WordSpace>
    <language>
      <grammar>complete.grammar</grammar>
    </language>
  </WordSpace>

  <InputSpace>
    <nDim>100</nDim>
    <nVectors>128</nVectors>
    <nOutput>128</nOutput>
    <nWhere>2</nWhere>
```

```
    <nWhen>2</nWhen>
    <lexer>sentence</lexer>
    <codebook>quantize</codebook>
  </InputSpace>

  <PerceptualSpace>
    <nOutput>128</nOutput>
    <nDim>100</nDim>
    <nVectors>128</nVectors>
    <hasAttention>false</hasAttention>
  </PerceptualSpace>

  <!-- ConceptualSpace: matches percept/symbol row count; state
mixing
       happens via iterative Sigma-Pi (not concatenation). -->
  <ConceptualSpace>
    <nOutput>128</nOutput>
    <nDim>100</nDim>
    <nVectors>128</nVectors>
    <nWhere>0</nWhere>
    <nWhen>0</nWhen>
    <hasAttention>true</hasAttention>
    <invertible>true</invertible>
    <tetralemmaOverride enabled="false" />
  </ConceptualSpace>

  <SymbolicSpace>
    <nOutput>128</nOutput>
    <nDim>100</nDim>
    <nVectors>128</nVectors>
    <nWhere>0</nWhere>
    <nWhen>0</nWhen>
    <accumulateTruth>0</accumulateTruth>
    <tetralemmaOverride enabled="false" />
  </SymbolicSpace>

  <OutputSpace>
    <nOutput>1</nOutput>
    <nDim>100</nDim>
    <nVectors>1</nVectors>
    <nWhere>0</nWhere>
    <nWhen>0</nWhen>
  </OutputSpace>

</model>



# STM Serial/Parallel Handling — Implementation Plan
```

Context

We are converging on a trainable model intended for head-to-head comparison with **nanochat** on FineWeb. The target config is data/MentalModel.xml — `<conceptualMode>serial</conceptualMode>`, `<modelType>embedding</modelType>`, `<hasAttention>true</hasAttention>` on `ConceptualSpace`, `<sentencePrediction>true</sentencePrediction>`, FineWeb data dir. The existing machinery is most of the way there (ShortTermMemory, the per-word loop `_forward_body_per_word`, the routing parser at `WordSubSpace.languageLayer`, `_chart_compose_at_C` / `_chart_generate_from_stm`, the lift/lower/union/intersection reverse contract, `InterSentenceLayer` ARMA over a `[S|W]` ring buffer at bin/Layers.py:5743, `TruthLayer` LTM with `truthMaxEntries` capacity gated on `truthMinMagnitude`). This plan gives it the slot-ordering contract, the trainable in-STM predictor, the per-word router wire-up, and the LTM / inter-sentence prediction surfaces needed for a coherent end-to-end loss.

Goal: a serial-mode trainable model where (a) each per-word step is **predict-then-perceive** with the prediction provided by the router-driven `IntraSentenceLayer`, (b) the routing parser fires per-word so its rule distribution is the prediction context, (c) reverse closes (deleting the WordSubSpace syntactic cache and replaying reverse from STM reconstructs words above a closeness threshold), (d) relative sentences (`part`, `isEqual`) leave the STM at depth 3 and are always inserted into the SS codebook with a tetralemma trust value graded against existing TruthSet luminosity, (e) LTM is the chain of post-reduction STM end-states, predicted forward by `InterSentenceLayer` lifted to the next-STM-shape regime.

Sequencing contract (the spec)

Both modes follow **predict-then-perceive**. Perception overwrites the slot the predictor just wrote — perception is ground truth; the prediction is the residual the loss reads against it.

Serial (per-word)

For each new word w_t:

1. **Shift STM** left-to-right by one slot so slot$[0]$ is free.

Slot$[\text{cap}-1]$ falls off the back.
2. **Predict** slot$[0]$ from slot$[1{:}\text{end}]$: $\hat c_t = \pi_\text{intra}(\text{STM}[1{:}\text{end}], \text{routing})$ where π_intra is the new `IntraSentenceLayer` and `routing` is the per-word router output.
3. **Perceive**: overwrite slot$[0]$ with the materialised CS event for w_t.
4. **Language** dispatch (SyntacticLayer cursors driven by `wordSubSpace.current_rules`) reads the layer.

Parallel (whole-slab)

In one shot:

1. **Predict** STM$_\text{new}[0{:}\text{end}]$ from STM$_\text{prev}[0{:}\text{end}]$: $\hat C = \pi_\text{intra}(C_\text{prev}, \text{routing})$.
2. **Perceive**: overwrite STM$_\text{new}[0{:}\text{end}]$ with the materialised CS slab.

The held $\hat c_t$ / $\hat C$ feeds the IR loss as the next-idea predictor target — directly analogous to a transformer LM's next-token loss but expressed in concept space. nanochat-style comparison is at this loss.

Architectural framing to document

Serial mode = attentional filtering

- **Serial mode is the mode in which attention narrows the per-word pipeline.** Parallel mode reads the whole slab; serial trades breadth for a focused beam. The existing serial-vs-attention guard at the top of bin/Models.py (which forces `conceptualSpace.serial_mode = False` when `hasAttention=True`) **is lifted** — MentalModel.xml runs serial **with** attention, and that is the regime we are documenting and training.
- **CS$\to$PS attention $\to$ meronymic mask.** A Gaussian envelope over word positions (existing `gaussian_window_word` at bin/Models.py:2275 is the kernel). Centring: peak at the current word, $\sigma = \text{maskRate} \cdot T$. Hardcoded for now. The doc records the **guidance-signal contract** so a future image-reading variant drops in a different centring policy without breaking callers.
- **CS$\to$SS attention $\to$ taxonymic mask.** When SS lookup needs a specific category (e.g. select NP from the taxonomy), the mask zeros out non-NP rows of the SS codebook *before* the lift/

lower lookup runs. Soft category prior expressed as a mask; once applied, the SS-side router only sees admissible rows.

Routing parser split: SS analysis vs CS execution

The router's two roles:

- **SS-side analysis** (the existing `WordSubSpace.compose` at bin/Language.py): soft superposition over the taxonymic codebook (with any codebook modifications — adding/grafting nodes when a relative sentence accepts a new edge). Output: routing distribution + selected rule list.
- **CS-side execution** (the existing `BinaryStructuredReductionLayer.forward` reduction + the per-tier `SyntacticLayer` cursors during reverse): applies the chosen reductions — `lift`, `lower`, `union`, `intersection`, `swap`, `quantize`, `not`. Only `lift` / `lower` / `union` / `intersection` need the codebook (inverse-recommended via `Ops.disjunctionReverse` / `Ops.conjunctionReverse`); `swap` / `quantize` / `not` are tensor-only.

The implementation is largely a doc + naming pass: the router code already separates routing from reduction. The plan locks the contract and adds the split-aware docstrings on `WordSubSpace.compose` / `WordSubSpace.generate`.

Masked-word reconstruction via priming

Prediction objective: sentence reconstruction. Masked words enter as the all-zeros vector (not in the PS codebook so identifiable post-hoc). The router fills blanks via a best-fit walk over the codebook biased by:

- The taxonymic prior (the CS$\to$SS mask is the route).
- The part-of-speech selection from the routing distribution.
- Accumulated prediction from previous stages (already-pushed STM slots + the held $\hat c_t$).

The priming machinery already exists (`left_priming` / `right_priming` in `Basis.lift` / `Basis.lower`, exercised by `test/test_primed_reverse_hard_mask.py` and `test/test_inverse_recommend.py`); we document that the priming source for the masked-word case is the `IntraSentenceLayer` prediction.

Relative vs absolute sentences

- **Absolute sentence**: reduces to a single idea in STM via lift/lower/conjunction/disjunction. End state: one non-zero slot.
- **Relative sentence**: `part(NP, NP)`, `isEqual(NP, NP)`,

`isEqual(NP, AP)` (rules already in MentalModel.xml at data/MentalModel.xml:68-70). Does **not** reduce. End state: three non-zero slots `[predicate, idea1, idea2]`.
- **Acceptance into the codebook — gated by `<truthCriterion>`, content-aware.** The existing `<truthMinMagnitude>` (a min-magnitude gate) and `<accumulateTruth>` (the SymbolicSpace per-sentence truth accumulator) are **both retired and replaced** by a single new knob `<truthCriterion>` $\in [0, 1]$:
  - At `truthCriterion = 1`: no new truths learned (highest bar).
  - At `truthCriterion = 0`: all proposed truths learned (no bar).
  - In between: a relative sentence is accepted into the SS codebook when its **learn-score** clears the criterion.
  - **Learn-score formula** (all three factors $\in [0, 1]$, product $\in [0, 1]$):
    $$\text{learn\_score} = \underbrace{\text{children\_in\_codebook}}_{\text{(idea1, idea2) already known}} \times \underbrace{\text{is\_truth\_obvious}}_{\text{agreement with existing knowledge}} \times \underbrace{\text{resolves\_contradiction}}_{\text{patches a known conflict}}$$
  - Accept the relation iff $\text{learn\_score} \ge \text{truthCriterion}$.
  - Component definitions:
    - `children_in_codebook` — fraction of `(idea1, idea2)` whose nearest-codebook-row distance is below an existing distance threshold (i.e. both children are already known concepts). Anchors new relations to existing knowledge instead of building taxonymic chains from thin air.
    - `is_truth_obvious` — agreement-with-existing-TruthSet score (computed from the existing luminosity / `truthBiasScale` machinery, normalised to $[0, 1]$). High when the relation is consistent with what we already believe.
    - `resolves_contradiction` — overlap between this relation's content and an unresolved contradiction in the TruthSet (i.e. there are two existing truths that this new relation would mediate between). High when adding the relation removes a known inconsistency.
  - **Lies and uncertain regions still get learned.** Low-`is_truth_obvious` (controversial / contradictory) relations are *not* gated out by the formula alone — if `children_in_codebook` and `resolves_contradiction` are high, the product can still clear the criterion. This is by design: we want to learn that people lie, and we want to learn in conflict-ridden areas.
  - **Accepted insertions carry a tetralemma trust value.** When `learn_score \ge truthCriterion`, the META edge `ss.insert_meta(predicate, idea1, idea2, trust=(t, f, b, n))` records a 4-tuple under existing `TetralemmaPolicy` ([data/

MentalModel.xml:233](data/MentalModel.xml:233)) computed from the relation's luminosity and its tetralemma posture against the TruthSet (TRUE / FALSE / BOTH / NEITHER weights, summing to 1). Downstream reasoning weighs the relation by its tuple, so a learned-but-low-trust relation contributes little to inference without being silently dropped.
  - **User-provided `<truth>` set is unchanged.** Truth accumulation reserved for user-provided gold truths stays separate; this gate only governs *learned* truths from training sentences.
- **Parallel mode**: STM is full (no reduction) — the whole slab is the end-state.

LTM as a chain of STM end-states

- **Definition**: LTM is the time-ordered concatenation of post-reduction STM end-states (1 entry per absolute sentence, 3 entries per relative sentence, the whole slab per parallel pass).
- **Carrier**: extend the existing `InterSentenceLayer` ring buffer at bin/Layers.py:5743 — today it stores `[S|W]` sentence reps; we add a parallel buffer for variable-shape STM end-state tuples (each tuple is `(depth, payload[depth, D])`). The existing `observe` / `cast` API gains an `observe_stm_end_state(tuple)` method.
- **Relation to TruthSet**: LTM is the **full chain**; TruthSet is the accepted-belief **subset** (gated by `truthMinMagnitude`). Both coexist; LTM drives prediction, TruthSet drives reasoning.

Inter-sentence prediction

- The same `IntraSentenceLayer` machinery, lifted one level: predicts the **shape of the next STM end-state** in conceptual space from the LTM tail.
- Wired into the existing `InterSentenceLayer.cast` path that already produces `_c_prior` (consumed at bin/Spaces.py:10573) — `cast` now emits a *shape*, not just a single prior vector. Tuple shape is predicted (1 vs 3 vs slab-width); content is predicted per-slot.
- Trains alongside the in-STM predictor with the same loss family.

Concrete code changes

1. Default STM length 9 $\to$ 8

- **bin/Layers.py:8375** — `DEFAULT_CAPACITY = 9` $\to$ `8`. Update the docstring's "default

```
9" mention at [bin/Layers.py:8366](bin/Layers.py:8366).
- **[data/model.xml](data/model.xml)** — if a default
`<stmCapacity>` is hardcoded, set to 8; if absent and inherits
from `DEFAULT_CAPACITY`, no change.
- **Doc**: update mentions in `doc/Params.md`, `doc/Spaces.md`,
and the docstrings of `_stm_shift_and_push` /
`_stm_set_all_slots` at [bin/Spaces.py:10214](bin/
Spaces.py:10214) and [bin/Spaces.py:10169](bin/Spaces.py:10169).

### 2. Explicit predict-then-perceive in CS.forward

Refactor [bin/Spaces.py:10398 ConceptualSpace.forward](bin/
Spaces.py:10398):

- Add two helpers next to `_stm_shift_and_push` /
`_stm_set_all_slots`:
  - `_stm_shift_for_predict()` — rotate slots so slot$[0]$ is
free (extracted from the rolling-window logic in
`_stm_shift_and_push`).
  - `_stm_run_intra_predict(prior_stm, routing)` — invoke
`self.intraSentenceLayer.forward(prior_stm, routing)` and stash
on `self._stm_predicted_idea` (serial) or
`self._stm_predicted_slab` (parallel). Does **not** mutate
buffer/depth.
- New control flow:
  - **Serial** (`folded.dim() == 2` or `folded.shape[1] == 1`):
    1. `snap = self.stm.snapshot()` (the old slots before shift).
    2. `self._stm_shift_for_predict()` — slot$[0]$ now free.
    3. `routing = self.wordSubSpace.current_rules` (just
populated by per-word router fire — see §4).
    4. `self._stm_run_intra_predict(snap[:, 1:], routing)` —
stashes $\hat c_t$.
    5. Write `idea` (materialised CS event for this word) into
slot$[0]$ via the existing buffer accessor.
  - **Parallel** (`folded.dim() == 3 and folded.shape[1] > 1`):
    1. `prev = self.stm.snapshot()` (may be empty on the first
pass; the predict step degenerates to identity then).
    2. `routing = self.wordSubSpace.current_rules` (sentence-
boundary fire).
    3. `self._stm_run_intra_predict(prev, routing)` — stashes $
\hat C$.
    4. `_stm_set_all_slots(folded)` (unchanged).

### 3. New `IntraSentenceLayer` — combined PI-then-Sigma, no
intermediate $\tanh$

New class at [bin/Layers.py](bin/Layers.py) (parallel naming to
`InterSentenceLayer`).
```

- **Architecture**: `pi(x) \to sigma(pi(x))` — a `PiLayer` immediately followed by a `SigmaLayer` with **no intermediate activation** (the two linear bodies fuse efficiently when there is no nonlinearity between them). PI first per the user's note — the PI body lifts low-rank STM slots into the working width, Sigma collapses the lifted slots into the predicted idea.
- **Signature**:
  ```python
  class IntraSentenceLayer(Layer):
      """In-STM autoregressive predictor: STM[1:end] -> predicted STM[0]
      (serial) / STM_prev[0:end] -> predicted STM_new[0:end] (parallel).
      Combined PI-then-Sigma, no intermediate tanh; the routing
      distribution from WordSubSpace.current_rules conditions the
      Sigma collapse (rule-aware predictor)."""
      def __init__(self, concept_dim, stm_capacity, routing_dim): ...
      def forward(self, prior_slots, routing): ...  # -> predicted [B, D] or [B, N, D]
      def reverse(self, predicted, routing): ...    # approximate inverse for the recon roundtrip
  ```
- **Routing conditioning**: the `routing` argument (the soft routing distribution from the per-word router) is injected as an additive bias on the Sigma collapse — concretely, project routing $\in \mathbb{R}^{\text{routing\_dim}}$ through a small linear to $\mathbb{R}^D$ and add to Sigma's pre-output. This makes the predictor rule-aware without a separate attention block.
- **Owned by `ConceptualSpace`**, built in `__init__` next to `_subspaceForPS` allocation at bin/Spaces.py:10121. Wired into the layer cascade so `Start()` / `Reset()` reach it.
- **Loss**: $\mathcal L_\text{intra} = \text{MSE}(\hat c_t, c_t)$ summed over per-word steps; in inference, $\hat c_t$ also conditions the masked-word recon priming. The loss is added to the existing IR-loss path with weight `<intraLossWeight>0.1</intraLossWeight>` (new knob under `<architecture><training>`).
- **Reverse contract**: `IntraSentenceLayer.reverse(predicted, routing) \approx prior_slots` — used by the reconstruction roundtrip in §6.

4. Per-word router firing (the in-STM predictor's conditioning context)

- **Forward**: inside the per-word loop body bin/Models.py:5766 (just before `_per_word_body_step` returns or right after — but **before** the CS event is written to STM), call

`self.wordSubSpace.compose(self.conceptualSpace.stm.snapshot())`. This populates `wordSubSpace.current_rules` for the SS dispatch path **and** provides the routing distribution that conditions `IntraSentenceLayer.forward`.
- **Reverse**: at the matching reverse-leg site, call `wordSubSpace.generate(snap)` per-word.
- **Sentence-boundary fire is retained** (`_chart_compose_at_C` at bin/Models.py:5848 before `_stm_reduce_to_single_S`) — it produces the final routing snapshot that the inter-sentence predictor consumes.
- **Gating knob**: `<routerWireSerial>per-word</routerWireSerial>` under `<architecture>`, values `{per-word, boundary, both, off}`. Default `both` (per-word during training so the intra-predictor sees routing context; boundary still fires for the inter-sentence predictor). Read in `BaseModel.__init__` next to `serial_mode` / `hasAttention` around bin/Models.py:590.
- **Lift the serial/attention guard**: remove the `if (m.serial_mode and getattr(m.conceptualSpace, 'hasAttention', False)): m.conceptualSpace.serial_mode = False` block. Add a comment recording the new doctrine: serial mode **is** the attentional-filtering regime.

5. SS-analysis / CS-execution split — doc + small refactor

- Audit `bin/Language.py` `WordSubSpace.compose` / `WordSubSpace.generate`:
  - Confirm inputs only read SS-side state (codebook + taxonomy mask + priming).
  - Confirm outputs are routing distribution + a hard rule list that CS executes.
- If `compose` calls any CS-side reduction directly, move that call to the CS-side path (the `BinaryStructuredReductionLayer.forward` already separates `routing` from reduction; this is a name + comment pass unless the audit surfaces a tangle).
- Add a docstring section to `WordSubSpace.compose`: *"This is the SS-side analysis stage. The CS-side execution stage runs in `ConceptualSpace.forward` (lift/lower/union/intersection/swap/quantize/not) and the per-tier `SyntacticLayer` cursors during reverse."*

6. reverse() invertibility audit + reconstruction-from-cleared-cache

Goal: deleting WordSubSpace's syntactic cache and running reverse should reconstruct the words that best match the held STM idea.

- New tests under `test/`:

  - `test_stm_reverse_roundtrip_lift_lower.py` — pin `Ops.lift` / `Ops.lower` for all (`mode`, `kind`) combos: `forward(reverse(y)) \approx y` and `reverse(forward(x)) \approx x` (using `test/test_ops_lift_lower.py` tolerances).
  - `test_stm_reverse_roundtrip_union_intersection.py` — pin `disjunctionReverse` / `conjunctionReverse`: codebook-fixed, priming-off, candidate-recommendation $\to$ forward closure.
  - `test_intra_sentence_layer_reverse.py` — pin `IntraSentenceLayer.reverse(forward(x)) \approx x` (the new layer needs its own roundtrip).
  - `test_stm_recon_from_cleared_cache.py` — full integration: run forward on a single MentalModel sentence, delete the WordSubSpace syntactic cache (`current_rules`, `generate_rules`, `recur_pass`), run reverse from the STM snapshot alone, assert top-k recovered words at each position overlap with input above the existing `atol=2e-1` closeness threshold.

7. Relative-sentence end-state + unconditional codebook insertion + trust value

- **Stop the bounded-reduce collapse at relative end-states.** In `_stm_reduce_to_single_S` at bin/Models.py:5850, check whether the top S-rule is a relative predicate (`part`, `isEqual` — identifiable by the grammar's `relative=True` marker added below). If yes, return the depth-3 `[predicate, idea1, idea2]` STM unchanged. Otherwise, collapse as today.
- **Grammar marker.** In the XML grammar parser (the path that reads `<rule>S = part(NP, NP)</rule>` etc.), tag the rules `part` / `isEqual` with `is_relative=True` on the parsed rule object. The check at the reduce site reads that tag.
- **Sentence-boundary insertion hook.** At the same site as `_chart_compose_at_C`, if the end-state is relative:
  1. Compute `learn_score` from the three factors above (helper `_compute_learn_score(predicate, idea1, idea2, truth_set)` on `ConceptualSpace`).
  2. If `learn_score < truthCriterion`, skip insertion (the relation may still be re-encountered later when the criterion is satisfied; nothing is permanently lost).
  3. Otherwise compute the tetralemma 4-tuple (t, f, b, n) from the relation's luminosity and posture against the TruthSet, and call `ss.insert_meta(predicate, idea1, idea2, trust=(t, f, b, n))` (extending the existing META taxonomy hook at [bin/Spaces.py:10312 _maybe_autobind_meta](bin/Spaces.py:10312) to take a trust kwarg).
- **Migration of retired knobs**:
  - `<truthMinMagnitude>` (default 0.3, currently in `<SymbolicSpace>` / `<ConceptualSpace>`) — **remove**. Its callers (bin/Spaces.py:11041, [bin/

Spaces.py:13321](bin/Spaces.py:13321), bin/Mereology.py:83, bin/Mereology.py:303, bin/Mereology.py:324) are audited and either deleted (the magnitude-gate is no longer how we accept) or rewritten to read `learn_score`'s `is_truth_obvious` component if they need an obviousness signal.
  - `<accumulateTruth>` (currently in `<SymbolicSpace>`, e.g. data/MentalModel.xml:258) — **remove**.
  - `<truthCriterion>` (new, default 0.3) — declared under `<architecture>` in data/model.xml and data/model.xsd; overridable per-space if needed.
- **New tests**:
  - `test_stm_relative_sentence_end_state.py` — `part(NP, NP)` $\to$ STM depth = 3, slots = `[part, idea1, idea2]`.
  - `test_relative_sentence_codebook_insertion.py` — after a relative sentence whose `learn_score` clears `truthCriterion`, `ss.taxonomy_children(predicate_pos)` includes the two ideas and the meta row carries a tetralemma 4-tuple.
  - `test_truth_criterion_gates_learning.py` — three cases: (a) `truthCriterion=1` $\to$ no insertion regardless of formula; (b) `truthCriterion=0` $\to$ insertion regardless of formula; (c) `truthCriterion=0.3` $\to$ insertion iff `learn_score >= 0.3`. Mock the three formula factors directly via test seam on `_compute_learn_score`.
  - `test_learn_score_components.py` — pin each factor in isolation: known children $\to$ `children_in_codebook=1`; perfectly-agreeing relation $\to$ `is_truth_obvious=1`; relation between two existing contradicting truths $\to$ `resolves_contradiction=1`.
  - `test_lies_can_be_learned.py` — a relation with low `is_truth_obvious` (i.e. contradicts existing TruthSet) but high `children_in_codebook` and high `resolves_contradiction` clears `truthCriterion=0.3` and lands in the codebook with a tetralemma tuple that records the conflict (high `b` weight).

8. LTM as a chain of STM end-states

- **Extend `InterSentenceLayer`** at bin/Layers.py:5743:
  - Add a second ring buffer `_stm_end_states` of size `<ltmCapacity>` (new knob, default 1024 — separate from TruthLayer's `<truthMaxEntries>` which now exclusively holds user-provided gold truths), storing `(depth, payload[depth, D], tetralemma)` tuples.
  - Add `observe_stm_end_state(tuple)` called from the sentence-boundary hook after the reduce / relative-preserve step decides the end-state. Note: every sentence's end-state lands in LTM regardless of `truthCriterion` — LTM is the AR sequence used for

inter-sentence prediction; truthCriterion only gates the separate SS-codebook insertion of *learned relations*.
  - Add `get_stm_chain(n=None)` accessor returning the last n end-states for downstream consumers.
- **No new top-level Layer** — LTM lives on `InterSentenceLayer` because the existing `cast` / `observe` machinery already runs at sentence boundaries.
- **New test**: `test_ltm_chain_grows.py` — process N sentences, assert `InterSentenceLayer.get_stm_chain(N)` returns N tuples with matching depths.

9. Inter-sentence prediction — extend `InterSentenceLayer.cast` to emit a shape

- Today's `cast` emits a single `_c_prior` vector. Change: `cast` now emits a predicted STM end-state shape `(depth_hat, payload_hat[depth_hat, D])` derived from running an `IntraSentenceLayer` instance over the `_stm_end_states` chain (the same predictor class, instantiated at the inter-sentence level with `stm_capacity = truthMaxEntries`).
- `BasicModel.generate_sentence` (bin/Models.py:2130) consumes the new shape: stage `payload_hat` per slot on `ConceptualSpace._c_prior` (extend the existing `_c_prior` to accept a `[depth, D]` tensor in addition to the current `[D]` / `[1, D]` shapes — see bin/Spaces.py:10573).
- **Loss**: $\mathcal L_\text{inter} = \text{MSE}(\hat{\text{payload}}, \text{payload})$ summed over sentences in the discourse window. Weight knob `<interLossWeight>0.1</interLossWeight>` next to `<intraLossWeight>`. nanochat-comparison reads at $\mathcal L_\text{intra} + \mathcal L_\text{inter}$.
- **New test**: `test_inter_sentence_prediction_shape.py` — observe a relative sentence (depth=3) followed by an absolute sentence (depth=1); assert `cast` after the relative emits a predicted shape (depth, payload) with depth $\in \{1, 3\}$ and finite payload.

10. Documentation deliverables

- **New `doc/STM.md`** — the chapter the rest of the docs cross-link to. Sections:
  1. STM data model (capacity, concept_dim, WordSubSpace data carrier).
  2. Serial sequencing (shift, predict, perceive, language).
  3. Parallel sequencing (predict, perceive).
  4. Attentional filtering: CS$\to$PS meronymic Gaussian mask, CS$\to$SS taxonymic mask; serial $=$ attention regime.
  5. Routing parser: SS-side analysis, CS-side execution.

  6. `IntraSentenceLayer`: combined PI-then-Sigma, routing-conditioned, the in-STM AR predictor.
  7. Per-word router firing and the prediction context.
  8. Masked-word reconstruction via priming.
  9. Relative vs absolute end-states; codebook insertion gated by content-aware `learn_score` against `<truthCriterion>`; tetralemma trust recorded on accepted insertions; user-provided `<truth>` set remains the separate gold-truth surface.
  10. LTM as the chain of STM end-states (on `InterSentenceLayer`).
  11. Inter-sentence prediction (lifted `IntraSentenceLayer` over the LTM chain).
  12. nanochat comparison framing: $\mathcal L_\text{IR} + \mathcal L_\text{intra} + \mathcal L_\text{inter}$ on FineWeb, MentalModel.xml config.
- **Add `doc/STM.md` to the Makefile chapter order** (between `doc/Spaces.md` and `doc/Language.md`).
- **Persist this plan to `doc/plans/2026-05-29-stm-serial-parallel-modes.md`** per the user's `feedback_plans_to_doc` memory.
- **Use LaTeX math, not Unicode glyphs** (`$\to$`, `$\hat c_t$`, `$\sigma$`, `$\mathcal L$`) per `feedback_latex_not_unicode`.

Files to modify (representative)

- bin/Layers.py:8366, bin/Layers.py:8375 — STM default capacity.
- bin/Layers.py:5743 — `InterSentenceLayer` extension: `_stm_end_states` ring buffer, `observe_stm_end_state`, `get_stm_chain`, `cast` returns shape.
- bin/Layers.py — new `IntraSentenceLayer` class.
- bin/Spaces.py:10121 — `ConceptualSpace.__init__` builds the `IntraSentenceLayer` and wires it into the cascade.
- bin/Spaces.py:10169, bin/Spaces.py:10214, bin/Spaces.py:10398 — predict-then-perceive helpers + `forward` refactor.
- [bin/Spaces.py:10312 _maybe_autobind_meta](bin/Spaces.py:10312) — extended to take a `trust=(t, f, b, n)` kwarg on `insert_meta`; new sibling helper `_compute_learn_score(predicate, idea1, idea2, truth_set)` and `_tetralemma_trust(relation, truth_set)` on `ConceptualSpace`.
- [bin/Spaces.py:11035-11044](bin/Spaces.py:11035), [bin/Spaces.py:13321-13322](bin/Spaces.py:13321), bin/Mereology.py:83, [bin/Mereology.py:303]

(bin/Mereology.py:303), bin/Mereology.py:324 — audit and either remove or rewrite the `_truth_min_magnitude` callers (the retired knob).
- bin/Spaces.py:10573 — `_c_prior` accepts `[depth, D]` tuple shapes from inter-sentence prediction.
- bin/Models.py:590 — `routerWireSerial` config read; serial/attention guard lifted.
- bin/Models.py:5766 — per-word router fire inside the per-word loop body.
- bin/Models.py:5848, bin/Models.py:5870, bin/Models.py:5887 — sentence-boundary router wire-up; reverse-leg mirror; relative-end-state preservation in `_stm_reduce_to_single_S`.
- bin/Models.py:2130 — `generate_sentence` consumes the new inter-sentence shape.
- bin/Language.py `WordSubSpace.compose` / `generate` — docstring split between SS analysis and CS execution.
- Grammar parser (the path that reads `<rule>...</rule>` from `<grammar><compose><symbols>`) — tag `part` / `isEqual` rules with `is_relative=True`.
- data/MentalModel.xml — add `<intraLossWeight>`, `<interLossWeight>`, `<routerWireSerial>both</routerWireSerial>`, `<truthCriterion>0.3</truthCriterion>`, `<ltmCapacity>1024</ltmCapacity>`. **Remove** `<accumulateTruth>` at data/MentalModel.xml:258. `<stmCapacity>` if explicit; otherwise inherits 8.
- data/model.xml, data/model.xsd — declare the new knobs with defaults; **remove** `<truthMinMagnitude>` and `<accumulateTruth>` (and any xsd schema entries for them).
- `test/test_intra_sentence_layer_reverse.py`, `test/test_stm_reverse_roundtrip_lift_lower.py`, `test/test_stm_reverse_roundtrip_union_intersection.py`, `test/test_stm_recon_from_cleared_cache.py`, `test/test_stm_relative_sentence_end_state.py`, `test/test_relative_sentence_codebook_insertion.py`, `test/test_truth_criterion_gates_learning.py`, `test/test_learn_score_components.py`, `test/test_lies_can_be_learned.py`, `test/test_ltm_chain_grows.py`, `test/test_inter_sentence_prediction_shape.py`, `test/test_router_fires_per_word.py` — new tests.
- `doc/STM.md` (new), `doc/Architecture.md`, `doc/Spaces.md`, `doc/Language.md`, `doc/Mereology.md`, `doc/Reasoning.md`, `doc/Params.md` — cross-link + update mentions (capacity, serial-with-attention doctrine, tetralemma trust value on relative meta edges).
- `Makefile` — add `doc/STM.md` to chapter order.

- `doc/plans/2026-05-29-stm-serial-parallel-modes.md` — persisted copy of this plan.

Verification

Per `feedback_targeted_tests`, run targeted node IDs, not the whole suite:

1. **STM default 8**: `pytest test/test_conceptual_stm.py -v -k capacity`.
2. **Serial sequencing**: `pytest test/test_serial_mode_conceptual.py -v` — no regression vs existing `atol=2e-1`; assert `self._stm_predicted_idea` populated after each per-word push.
3. **Parallel sequencing**: `pytest test/test_hierarchical.py test/test_xor_exact.py -v`.
4. **IntraSentenceLayer roundtrip**: `pytest test/test_intra_sentence_layer_reverse.py -v`.
5. **Router fires per word**: `pytest test/test_router_fires_per_word.py -v` — call-count > 0 per word in serial.
6. **Reverse roundtrip ops**: `pytest test/test_stm_reverse_roundtrip_lift_lower.py test/test_stm_reverse_roundtrip_union_intersection.py -v`.
7. **Reconstruction from cleared cache**: `pytest test/test_stm_recon_from_cleared_cache.py -v`.
8. **Relative-sentence end-state + truthCriterion**: `pytest test/test_stm_relative_sentence_end_state.py test/test_relative_sentence_codebook_insertion.py test/test_truth_criterion_gates_learning.py test/test_learn_score_components.py test/test_lies_can_be_learned.py -v`.
9. **LTM chain**: `pytest test/test_ltm_chain_grows.py -v`.
10. **Inter-sentence prediction**: `pytest test/test_inter_sentence_prediction_shape.py -v`.
11. **Smoke gates (no regressions)**:
    - `make xor` (`MM_xor`) green.
    - Active-payload-audit green.
    - `recon-breaks-MM_5M`-cluster xfails: confirm no new failures (and note whether any side-effect-clear).
12. **End-to-end doc build**: `make doc` succeeds (catches pandoc/xelatex Unicode-glyph trap in `doc/STM.md`).
13. **MentalModel training smoke**: `make train MODEL=MentalModel` runs one epoch on a small FineWeb shard without NaN/Inf loss (per `feedback_fail_loud_on_divergence` — NaN must raise loudly, not be silenced).
14. **nanochat comparison setup** (documented, not run as part of

this plan's CI): a separate evaluation harness reads $\mathcal L_\text{IR} + \mathcal L_\text{intra} + \mathcal L_\text{inter}$ on a FineWeb held-out split and compares against published nanochat numbers at matched parameter count.

Out of scope

- The nanochat **comparison evaluation harness itself** (reading nanochat checkpoints, matching tokenizers, running on the same held-out split). The plan delivers the trainable model + the loss that makes the comparison meaningful; the harness is a separate plan.
- **Learning** the CS$\to$PS Gaussian mask centring / σ (still hardcoded). The doc captures the guidance-signal contract so a learned variant drops in.
- **Per-stage `<intraLossWeight>` / `<interLossWeight>` tuning** beyond the initial 0.1 defaults — leave to a hyperparameter sweep after the first training run.

```
<?xml version="1.0"?>
<grammar name="complete">

  <compose>
    <rule>S = sigma(S)</rule>
    <rule>S = NP</rule>
    <rule>S = lift(NP, VP)</rule>
    <rule>S = isEqual(NP, NP)</rule>    <!-- marker: "is" -->
    <rule>S = isEqual(NP, AP)</rule>    <!-- marker: "is" -->
    <rule>S = part(NP, NP)</rule>    <!-- markers: "has", "is-a"
-->
    <rule>S = not(S)</rule>    <!-- marker: "not" -->
    <rule>S = conjunction(S, S)</rule>    <!-- marker: "and" -->
    <rule>S = disjunction(S, S)</rule>    <!-- marker: "or" -->
    <rule>S = query(NP, AP)</rule>
    <rule>S = query(NP, NP)</rule>
    <rule>NP = N</rule>
    <rule>NP = AP</rule>
    <rule>NP = conjunction(NP, NP)</rule>    <!-- marker: "and"
-->
    <rule>NP = disjunction(NP, NP)</rule>    <!-- marker: "or" --
>
    <rule>VP = V</rule>
    <rule>VP = lower(ADV, VP)</rule>
    <rule>VP = lift(MP, VP)</rule>
    <rule>VP = lower(ADJ, VP)</rule>
```

```
    <rule>VP = intersection(V, O)</rule>
    <rule>O = NP</rule>
    <rule>AP = lower(ADJ, NP)</rule>
    <rule>AP = lower(ADJ, AP)</rule>
    <rule>AP = lower(DET, NP)</rule>
    <rule>MP = ADV</rule>
    <rule>MP = ADV MP</rule>
    <rule>PP = lift(P, NP)</rule>
    <rule>NP = lower(NP, PP)</rule>
    <rule>VP = lower(VP, PP)</rule>
    <rule>S = true(S)</rule>
    <rule>S = false(S)</rule>
    <rule>S = non(S)</rule>
    <rule>S = intersection(S, S)</rule>
    <rule>S = union(S, S)</rule>
    <rule>S = swap(S, S)</rule>
  </compose>

  <generate>
    <rule>NP = S</rule>
    <rule>NP, VP = lift.reverse(S)</rule>
    <rule>NP, EQUALS_MARK, NP = isEqual.reverse(S)</rule>
    <rule>NP, EQUALS_MARK, AP = isEqual.reverse(S)</rule>
    <rule>NP, PART_MARK, NP = part.reverse(S)</rule>
    <rule>NOT_MARK, S = not.reverse(S)</rule>
    <rule>NON_MARK, S = non.reverse(S)</rule>
    <rule>S, CONJ_MARK, S = conjunction.reverse(S)</rule>
    <rule>S, DISJ_MARK, S = disjunction.reverse(S)</rule>
    <rule>EQUALS_MARK, NP, AP, QUERY_MARK = query.reverse(S)</
rule>
    <rule>EQUALS_MARK, NP, NP, QUERY_MARK = query.reverse(S)</
rule>
    <rule>NP, CONJ_MARK, NP = conjunction.reverse(NP)</rule>
    <rule>NP, DISJ_MARK, NP = disjunction.reverse(NP)</rule>
    <rule>ADV, VP = lower.reverse(VP)</rule>
    <rule>MP, VP = lift.reverse(VP)</rule>
    <rule>ADJ, VP = lower.reverse(VP)</rule>
    <rule>V, O = intersection.reverse(VP)</rule>
    <rule>ADJ, NP = lower.reverse(AP)</rule>
    <rule>ADJ, AP = lower.reverse(AP)</rule>
    <rule>DET, NP = lower.reverse(AP)</rule>
  </generate>
</grammar>
```